SMART INDIA HACKATHON 2026 · PS SIH26143 · FULL-STACK BUILD PLAN

SAGARNETRA

A maritime pollution intelligence platform. It finds oil slicks in satellite radar, works out where the oil came from, pulls every ship that was there, and hands a pollution officer a ranked, explainable list of suspects.

```
SAR scene → U-Net detects slick → geolocate + drift backtrack → AIS ships
in the zone → categorise + rank → officer evidence pack
```

IMAGERY	MODEL	SHIP DATA	HOSTING
Sentinel-1 C-band SAR, VV	**Custom U-Net, 5 classes**	**AIS via AISStream**	**Vercel · Render · Atlas**

The pipeline, stage by stage

Everything in the app exists to move an incident through these six stages. Each stage is one backend module and one screen.

1 Watch
new Sentinel-1 scene over a watch zone

2 Detect
U-Net mask → polygon, area, centroid, time

3 Backtrack
drift model → origin zones at t-6h, t-12h, t-24h

4 Ships nearby
AIS tracks inside zones during the window

5 Categorise + rank
type, behaviour, AIS gaps, explainable risk score

6 Officer
evidence pack, track replay, inspection, PDF report

STAGE	WHAT HAPPENS	REAL OR SIMULATED IN THE DEMO
1 Watch	Scheduler polls Copernicus Data Space for new Sentinel-1 GRD scenes intersecting saved watch zones, clips to the zone, cuts 256 px tiles.	Real API. Demo uses a "New scene received" button that pushes a bundled tile through the same code.
2 Detect	U-Net segments each tile. Mask is cleaned, look-alike pixels suppressed, largest blob traced to a polygon, pixels converted to lon/lat.	Real model, trained by us on the Krestenitis benchmark.
3 Backtrack	Wind and current fetched for the slick location. Centroid integrated backwards. Three origin ellipses produced.	Real weather data from Open-Meteo. Our own simple drift model.
4 Ships nearby	Query stored AIS positions inside the origin zones during the time window. Attach vessel identity and full track.	Live feed is real. Historical tracks for the demo incident are a generated scenario, stated openly.
5 Categorise + rank	Each ship gets a type, a behaviour label and a risk score from eight weighted features. Split into three tiers.	Real code, deterministic, every number explained on screen.
6 Officer	Evidence pack, track replay, notes, inspection actions, PDF export, source hashes for chain of custody.	Real.

Users and logins

This is a government tool, so there is no public signup. An administrator provisions accounts. Two roles cover everything the app does: the admin keeps the platform running, the officer works the cases.

ROLE	WHO	CAN DO
Admin	System owner, Coast Guard IT lead, one or two per deployment	Create, reset and disable officer accounts. Draw and edit watch zones. Read the audit log. Check system health: model, database, last scene per zone, AIS collector. Run seeding. Plus everything the officer can do.
Officer	Coast Guard pollution response officer, the main user	Verify detections, open incidents, review the ranked suspects, replay tracks, add notes, mark vessels for inspection, request port state control, export the PDF evidence pack, use the assistant, watch live traffic.

How authentication works

1. Officer enters email and password on `/login`. The frontend posts to `POST /api/auth/login`.
2. Backend looks up the user in the `users` collection, checks the bcrypt hash, updates `last_login`.
3. Backend returns a JWT signed with HS256 containing `sub`, `role`, `exp`. Expiry is 12 hours.
4. Frontend stores the token in localStorage, puts the user in the AuthProvider context, and redirects to `/app`.

5. Every API call carries `Authorization: Bearer <token>`. Any 401 clears the token and returns the user to `/login`.
6. Protected routers use `Depends(require_role("officer"))`. The dependency decodes the token, loads the user, checks the role, and hands the user object to the handler.

ENDPOINTS	`POST /api/auth/login` · `GET /api/auth/me` · `POST /api/auth/logout` · `GET/POST /api/admin/users` · `PATCH /api/admin/users/{id}`
HASHING	The `bcrypt` package directly, cost 12. Not passlib, which is unmaintained.
SECRET	32 random bytes in `JWT_SECRET`, never in the repo. Rotating it logs everyone out, which is fine.
ABUSE	`slowapi` limits login to 5 attempts per minute per IP. CORS allowlist holds only the Vercel domain and localhost.
AUDIT	Every detection, incident change and report export stores `created_by` and a timestamp. The admin page lists them.
SEEDS	`admin@sagarnetra.in` and `officer@sagarnetra.in` created by the seed script. The login page has a "Sign in as demo officer" button that fills them, so nothing blocks on stage.
LATER	Refresh token rotation and OTP are the production upgrades. Say so when a judge asks; do not build them now.

Features by page

Ten screens. Each one owns one job. Landing and Login are public; everything under `/app` sits behind RequireAuth. The officer uses eight of them; the Admin page is the admin's alone.

ROUTE	PAGE	WHAT THE USER SEES AND DOES	DATA BEHIND IT
`/`	Landing	Problem statement, the six-stage pipeline, one screenshot per pillar, "Open console" button.	Static
`/login`	Login	Email, password, demo officer button, error messages that say what went wrong.	`/api/auth/login`
`/app`	Dashboard	Open incidents, slicks detected this month, total area, watch zones with last scene date, recent activity feed, small trend chart.	`/api/overview`
`/app/detect`	Detection Console	Pick a bundled Sentinel-1 tile or upload one. Run the model. See the SAR image, the mask overlay with per-class colours, the traced polygon, area in km², confidence, inference time, and whether the U-Net or the heuristic fallback answered. "Create incident" sends it down the pipeline.	`/api/detect`, `/api/detect/samples`
`/app/map`	Live Map	Cesium globe. Slick polygons, three drift origin ellipses, live AIS vessels as oriented markers coloured by type, watch zones. Click a vessel for its card. Time slider replays the incident window.	`/api/incidents`, `/api/vessels/live`, `/api/incidents/{id}/tracks`
`/app/incidents`	Incidents	Table of every incident with status, area, tier of top suspect, assigned officer. Filters by zone, date, status.	`/api/incidents`
`/app/incidents/:id`	Investigation	The evidence pack. Left: SAR image and mask, slick facts, drift assumptions. Centre: ranked vessel list with tier chips and feature bars. Right: selected vessel's identity, track replay, AIS gaps highlighted. Actions: mark for inspection, request port state control, add note, export PDF.	`/api/incidents/{id}`, `/api/incidents/{id}/ranking`
`/app/vessels`	Vessels	Live AIS table for the watch zones. Search by name or MMSI. Type and flag filters. Click	`/api/vessels/live`, `/api/vessels/{mmsi}`

ROUTE	PAGE	WHAT THE USER SEES AND DOES	DATA BEHIND IT
		through to the vessel on the map.	
`/app/reports`	Reports	List of generated reports, regenerate, download PDF. A report contains the SAR image, polygon map, drift assumptions, ranking table with feature breakdown, officer notes and source hashes.	`/api/reports`
`/app/admin`	Admin	Four tabs. Users: create, reset password, disable. Watch zones: draw and edit monitored areas. Audit log: who ran, changed or exported what. System health: model engine, database, last scene per zone, AIS collector status. Admin role only; the sidebar link is hidden for officers.	`/api/admin/*` , `/api/health`

The assistant is a slide-over drawer available on every `/app` page. It answers questions about the current incident from the data, for example "why is MV Kaveri ranked first" or "how big is the slick in football fields". Built last.

How every module is built

Each block below is one folder in the repo with one owner. The stack line is the exact set of libraries. Nothing here is optional except where marked.

Frontend shell

`React 18 · TypeScript · Vite · Tailwind · react-router · Zustand · TanStack Query`

What. A single-page app with a public landing and login, and an authenticated shell with a left sidebar, top bar showing the user and role, and the ten pages above.

How. Vite project in `frontend/`. Tailwind for layout with a small token file for the palette. Zustand holds UI state: selected incident, map camera, time slider. TanStack Query wraps every API call so pages get loading, error and cache behaviour for free. A typed `api.ts` client reads `VITE_API_BASE`, attaches the bearer token, and handles 401. Recharts for the dashboard chart, Framer Motion for the console's mask reveal, lucide-react icons.

Done when. Every route renders with mock data before the backend exists, using MSW handlers that mirror the API contract.

3D map

`Cesium · vite-plugin-cesium · GeoJSON`

What. The globe on Live Map and the small map in Investigation.

How. One `MapView` component owns the Cesium viewer. Base imagery from OpenStreetMap or Sentinel-2 cloudless via a plain tile URL, so no Cesium ion token is required. Layers are separate hooks: `useSlickLayer` draws polygons from GeoJSON, `useDriftLayer` draws the three ellipses with decreasing opacity, `useVesselLayer` draws billboard markers rotated by heading and coloured by type, `useTrackLayer` draws polylines for the selected vessel and animates a point along it with the time slider. Picking uses `scene.pick` and dispatches into Zustand.

Done when. A demo incident shows slick, ellipses and eight vessels, and dragging the slider moves the ships.

Backend API

`Python 3.12 · FastAPI · uvicorn · pydantic v2 · Motor · httpx · APScheduler`

What. One service in `backend/` exposing the API surface below, running the scheduler, and calling the ML package in-process.

How. `app/main.py` creates the app, mounts routers, opens the Mongo client on startup, ensures indexes, starts APScheduler. Routers in `app/routers/` : auth, admin, overview, detect, incidents, vessels, reports, assistant. Business logic lives in `app/services/` , not in routers. Settings via `pydantic-settings` from environment. Auto-generated docs at `/docs` are shown to judges. A `/api/health` endpoint reports database and model status.

Done when. `pytest` passes against an in-memory Mongo, and `/docs` lists every endpoint with schemas.

Authentication

`PyJWT · bcrypt · slowapi`

What. The login flow described in the section above.

How. `app/security.py` has `hash_password` , `verify_password` , `create_token` , `decode_token` . `app/deps.py` has `get_current_user` and `require_role(*roles)` . Roles are an enum. The frontend has `AuthProvider` , `useAuth` , `RequireAuth` and a `RequireRole` wrapper that hides the admin page and link from officers. Roles are nested: admin includes everything officer can do.

Done when. Wrong password returns 401 with a clear message, an officer calling an admin endpoint gets 403, an expired token bounces to login.

Database

`MongoDB Atlas M0 · Motor · 2dsphere indexes`

What. Eight collections listed in the data model. GeoJSON geometry on detections, incidents, AIS positions and watch zones so spatial queries run in the database.

How. `app/db.py` holds the client and an `ensure_indexes` function: unique on `users.email` , 2dsphere on every `geometry` field, compound on `ais_positions (mmsi, ts)` , TTL of 7 days on live AIS positions so the free tier never fills. Local development uses Docker Compose with a Mongo container. A `seed_db.py` script writes the admin, officer, one watch zone, the demo scenario and its tracks.

Why Mongo and not PostGIS. Detection documents vary in shape, 2dsphere covers the queries we need, and Atlas M0 is free. Know this answer for the judges.

Computer vision model

`PyTorch · albumentations · torchmetrics · ONNX · onnxruntime · OpenCV`

What. A U-Net we write ourselves that labels every pixel of a Sentinel-1 tile as sea, oil spill, look-alike, ship or land.

Data. Krestenitis et al. 2019, requested from MKLab. 1,112 images at 1250 × 650 with 5-class masks, split 1,002 train and 110 test. Tiles are cut to 256 × 256 with overlap. Augmentations: flips, 90° rotations, brightness and speckle noise.

Architecture. `ml/unet.py`. Four encoder blocks with channels 32, 64, 128, 256, a 512 bottleneck, mirrored decoder with skip connections, batch norm, 1 × 1 head to 5 classes. About 7.8 million parameters. Small enough for CPU inference under a second per tile.

Training. `ml/train.py` run on a free Colab T4. Loss is class-weighted cross-entropy plus Dice, because sea is over 90 percent of pixels. AdamW, one-cycle learning rate, 40 epochs, about an hour. Metric is per-class IoU and mIoU on the 110 test images, saved to `ml/metrics.json` and shown on the Detection Console.

Serving. `ml/export.py` exports to ONNX. `ml/infer.py` loads it with onnxruntime, runs softmax, takes the oil class, removes blobs under 40 pixels, dilates once, and traces contours with OpenCV. Confidence is the mean oil probability inside the final mask. Returns mask PNG, pixel polygon, pixel area, confidence, and which engine answered.

Fallback. If the model file is missing or fails, `ml/heuristic.py` runs adaptive thresholding for dark spots plus morphology. The API marks the response `engine: "heuristic"` and the console shows it in amber. This is a safety net and an ablation, never hidden.

Optional. Train a second model with segmentation_models_pytorch and a ResNet18 encoder as a comparison row in the metrics table.

Scene ingest and watch zones

`Copernicus Data Space OData API · rasterio · APScheduler`

What. The automation that makes stage 1 real.

How. A watch zone is a polygon with a name, saved by the admin. Every 6 hours the scheduler queries Copernicus for Sentinel-1 GRD IW products intersecting each zone since its `last_checked`. For a new product it downloads only the VV measurement band with a windowed read, clips to the zone, scales to 8-bit with a fixed dB stretch, tiles to 256 px with georeference kept, and queues each tile for detection. Free Copernicus account, token in `CDSE_TOKEN`.

Demo. Four real Sentinel-1 tiles over the Gujarat and Mumbai offshore shipping lanes are bundled in `ml/samples/` with their bounding boxes. The console's "New scene received" button calls the same `process_tile` function the scheduler calls.

Georeferencing

`rasterio · shapely · pyproj`

What. Turning pixel results into geography.

How. `ml/geo.py`. If the tile is a GeoTIFF, the affine transform from rasterio maps pixel corners to lon/lat. If it is a plain PNG sample, the stored bounding box is interpolated. The pixel ring becomes a shapely polygon, is simplified with a 1-pixel tolerance, and closed. Area is computed after projecting to the local UTM zone with pyproj, never in degrees. Centroid, long-axis heading from the minimum rotated rectangle, and bounding box are returned as GeoJSON.

Drift backtracking

`Open-Meteo Marine API · Open-Meteo Forecast API · NumPy`

What. Where the oil was 6, 12 and 24 hours before the image was taken.

How. `services/drift.py`. Fetch hourly surface current u and v from Open-Meteo Marine and 10 m wind u and v from Open-Meteo Forecast for the slick centroid across the window. Both are free with no key and have a historical endpoint. Slick velocity is current plus 3 percent of wind, the standard leeway coefficient. Integrate the centroid backwards in 15-minute steps. Uncertainty grows as an ellipse with semi-axes of 0.5 km per hour along the drift direction and 0.3 km per hour across it. Output three GeoJSON ellipses tagged with their hour. The slick's long-axis heading is passed along as the likely vessel course.

Honest limit. This is a first-order model. OpenDrift is the research-grade alternative and is the stated upgrade path.

AIS ingestion

`AISStream.io websocket · websockets · Motor · scenario generator`

What. Live positions for the Vessels page and stored tracks for correlation.

How, live. `services/ais_collector.py` keeps one websocket to AISStream subscribed to the bounding boxes of all watch zones. Position reports are upserted into `ais_positions` with a 7-day TTL. Static reports update `vessels` with name, IMO, type, dimensions, flag from MMSI prefix, destination and draft. Free API key in `AISSTREAM_API_KEY`.

How, history. AISStream has no history. `scripts/gen_scenario.py` generates a realistic 36-hour scenario: eight vessels on plausible lanes, one tanker that switches AIS off for 40 minutes while crossing the drift origin, slows, and shows a draft decrease. The scenario writes to the same collections the collector writes to, so the correlation code cannot tell them apart. The Investigation page labels the incident "demo scenario". The collector runs continuously from deployment day so a few weeks of real positions exist by the finale.

Categorisation and risk ranking

`Plain Python · NumPy · shapely`

What. Stage 5. Deterministic, explainable, no black box.

Candidates. Every vessel with at least one position inside any origin ellipse between 30 hours before and the acquisition time, plus any vessel whose track segment crosses an ellipse even if no report fell inside it, which catches AIS gaps.

Categorise. Type from AIS ship type code grouped into tanker, cargo, fishing, passenger, tug, other. Behaviour from the track: transiting, loitering under 2 knots for over an hour, anchored, dark if any gap over 20 minutes inside the window.

FEATURE	COMPUTED AS	WEIGHT
Spatial fit	1 minus normalised closest approach of the track to the best-matching origin ellipse	25
Temporal fit	Minutes inside the ellipse during its time window, capped	15
AIS gap	Longest gap inside the window, minutes over 20, capped at 120	20
Heading match	Cosine similarity between vessel course near the zone and the slick's long axis	10
Manoeuvre	Speed drop over 3 knots or course change over 30° within the zone	10
Draft change	Reported draft decrease over the window, tankers only	8
Type prior	Tanker 1.0, cargo 0.7, passenger 0.4, tug 0.3, fishing 0.2	8
History	Prior incidents in our database for the same MMSI	4

Score is the weighted sum on a 0 to 100 scale. Tiers: `prime suspect ≥ 65` `person of interest 35 to 64` `cleared < 35`. The response includes every feature's raw value, normalised value and contribution, and the UI draws them as bars. Weights live in one config file so they can be tuned in a minute.

Investigation and incidents

`FastAPI · Motor · React`

What. The incident lifecycle: detected, under investigation, inspection requested, closed.

How. Creating an incident from a detection runs drift and ranking once and stores the results, so the page loads instantly and the numbers never change under the officer. "Re-run ranking" recomputes on demand. Notes, status changes and inspection requests append to an `events` array with user and time. SHA-256 of the source tile and the mask PNG are stored at creation and printed in the report.

Assistant

`Anthropic SDK · claude-sonnet-5 · tool use`

What. A drawer where the officer asks questions in plain language about the open incident.

How. Server-side only in `routers/assistant.py`. The system prompt explains the platform and the ranking features. Three tools: `get_incident`, `get_ranking`, `get_vessel`. The model calls tools, the server executes them against Mongo, and the answer streams back. The model never sees raw AIS beyond the current incident. If `ANTHROPIC_API_KEY` is absent, the drawer shows "Assistant unavailable in this deployment" and nothing else breaks. Built in the last week.

Reports

`React print stylesheet · html2pdf.js`

What. A PDF the officer attaches to an inspection request.

How. A `/app/reports/:id/print` route renders the report as a clean A4 page: SAR image and mask, map snapshot captured from Cesium's canvas, slick facts, drift assumptions, ranking table with feature bars, officer notes, source hashes, generated-by and timestamp. html2pdf.js renders it to PDF in the browser. The backend stores report metadata so the Reports page lists them.

Infrastructure

`GitHub monorepo · GitHub Actions · Docker · Vercel · Render · MongoDB Atlas`

What. One repo, two deployments, one database.

How. Frontend deploys to Vercel from `frontend/` on every push to main with `VITE_API_BASE` set to the Render URL. Backend deploys to Render as a Docker service from `backend/Dockerfile` on `python:3.12-slim` with onnxruntime, not torch, so the image stays under 600 MB and memory under 400 MB. Start with the Render starter plan to avoid cold starts on stage. Atlas M0 cluster with IP allowlist open and a dedicated database user. GitHub Actions runs ruff, pytest, tsc and vite build on every pull request.

ENV	`MONGODB_URI` · `JWT_SECRET` · `CORS_ORIGINS` · `AISSTREAM_API_KEY` · `CDSE_TOKEN` · `ANTHROPIC_API_KEY` optional · `VITE_API_BASE` on Vercel

---

UI direction

An operations console, not a marketing site. Officers use it in an ops room, often at night, with radar imagery and a globe on screen. The app is dark-first but restrained: deep navy ground, cool grey-blue panels, one amber accent reserved for alerts and suspects, teal for sea and normal state. Radar tiles are greyscale, so the

palette stays quiet and lets the imagery read. The landing page is lighter and more editorial because judges see it first.

ELEMENT	DECISION
Headings	Barlow Semi Condensed. Reads as instrumentation and saves width inside panels.
Body	Source Sans 3 or IBM Plex Sans.
Data	IBM Plex Mono with tabular numerals for MMSI, IMO, coordinates, timestamps, area, confidence. Units on every number. Coordinates as decimal degrees to four places with a copy button. Timestamps in UTC with IST on hover.
State as form	Tier gets a chip, an icon and a colour: red prime suspect, amber person of interest, green cleared. Engine gets a badge: teal U-Net or amber Heuristic. Incident status is a left stripe on its row. Colour is never the only signal.
Shell	Collapsible 240 px sidebar with icons and labels. Top bar with the active watch zone, three live status dots for model, database and AIS feed, and the user menu showing the role. Map screens have no page-level scroll.
Components	Tailwind with a token file for the palette. shadcn/ui for dialogs, tabs, tables, sliders and the command palette. lucide-react icons. Recharts for the two charts. Framer Motion only for the mask reveal, the drift ellipse fade and list reorder. Cmd+K jumps to any vessel or incident.

Key screens

SCREEN	LAYOUT
Login	Split. Left half a real Sentinel-1 tile with a faint slick, right half the form with the demo officer button under the password field.
Dashboard	Four KPI tiles, then two columns: open incidents with tier chips, and watch zone cards with last scene date and ships tracked. Trend chart and activity feed below.
Detection Console	Three panels. Left: bundled tile thumbnails and an upload drop zone. Centre: the SAR image on a canvas with mask overlay, opacity slider, per-class legend and traced polygon. Right: results card with area, confidence, engine badge, class pixel breakdown, model mIoU and the single Create incident button. The mask fades in when inference returns.
Live Map	Full-bleed globe. Floating panels only: layer toggles top left, legend bottom left, time slider bottom centre, vessel card top right on click.
Investigation	Three columns. Evidence left: tile, mask, slick facts, drift assumptions. Ranked list centre: tier chip, name, type, score, eight small feature bars per row. Vessel detail right: identity, mini map replaying the track, timeline strip with the AIS gap as a red interval.
Vessels	Dense table with sticky header, search by name or MMSI, type and flag filters, click through to the map.
Reports	List, plus an A4 print view on white so the PDF is legible.
Admin	Four tabs: Users, Watch zones with a small map to draw polygons, Audit log, System health.

Data model

COLLECTION	KEY FIELDS	INDEXES
`users`	email, password_hash, name, role, org, active, created_at, last_login	unique email
`watch_zones`	name, geometry (Polygon), last_checked, created_by	2dsphere geometry
`scenes`	product_id, acquired_at, footprint, zone_id, tiles_processed, source	product_id, acquired_at
`detections`	scene_id, tile_id, geometry (Polygon), centroid, area_km2, heading_deg, confidence, engine, mask_png, class_pixels, created_by, created_at, tile_sha256	2dsphere geometry, created_at
`incidents`	detection_id, status, zone_id, origin_zones [3 ellipses], drift_inputs, ranking [per vessel], assigned_to, events [], is_demo, created_at	status, created_at, 2dsphere origin_zones.geometry
`vessels`	mmsi, imo, name, callsign, type_code, type_group, flag, length, width, destination, last_seen	unique mmsi, name text
`ais_positions`	mmsi, ts, geometry (Point), sog, cog, heading, nav_status, draft, source	(mmsi, ts), 2dsphere geometry, TTL 7 days on live source
`reports`	incident_id, generated_by, generated_at, pdf_size, hashes	incident_id

API surface

METHOD AND PATH	ROLE	PURPOSE
`POST /api/auth/login`	public	Returns token and user
`GET /api/auth/me`	any	Current user
`GET /api/health`	public	Database, model and collector status
`GET /api/overview`	officer	Dashboard numbers and recent activity
`GET /api/detect/samples`	officer	Bundled Sentinel-1 tiles with bboxes
`POST /api/detect`	officer	Multipart image or sample_id → mask, polygon, area, confidence, engine. Persists a detection.
`POST /api/incidents`	officer	From detection_id. Runs drift and ranking. Returns incident.
`GET /api/incidents`	officer	List with filters
`GET /api/incidents/{id}`	officer	Full evidence pack
`GET /api/incidents/{id}/tracks`	officer	Candidate vessel tracks for replay
`POST /api/incidents/{id}/rerank`	officer	Recompute ranking
`POST /api/incidents/{id}/events`	officer	Note, status change, inspection request
`GET /api/vessels/live`	officer	Latest position per vessel inside watch zones
`GET /api/vessels/{mmsi}`	officer	Identity and recent track
`GET /api/reports` · `POST /api/reports`	officer	List and register generated reports
`POST /api/assistant/chat`	officer	Streams an answer grounded in the incident
`GET/POST /api/admin/users` · `PATCH /api/admin/users/{id}`	admin	User provisioning
`GET/POST /api/admin/zones`	admin	Watch zones
`GET /api/admin/audit`	admin	Who did what

Repository layout

```
sagarnetra/
├── frontend/                React + Vite, deploys to Vercel
│   ├── src/auth/            AuthProvider, RequireAuth, RequireRole
│   ├── src/lib/api.ts       typed client, bearer header, 401 handling
│   ├── src/map/             MapView + layer hooks (slick, drift, vessels, tracks)
│   ├── src/pages/           Landing, Login, Dashboard, DetectionConsole, LiveMap,
│   │                        Incidents, Investigation, Vessels, Reports, Admin
│   ├── src/store/           Zustand slices
│   └── public/sar/          bundled sample tiles for the console
├── backend/                 FastAPI, deploys to Render (Docker)
│   ├── app/main.py          app factory, routers, scheduler, indexes
│   ├── app/routers/         auth, admin, overview, detect, incidents, vessels, reports,
assistant
│   ├── app/services/        drift.py, ranking.py, ais_collector.py, ingest.py,
incidents.py
│   ├── app/security.py      hashing + JWT       app/deps.py  get_current_user,
require_role
│   ├── app/db.py            Motor client + ensure_indexes
│   ├── scripts/             seed_db.py, gen_scenario.py, verify_api.py
│   ├── tests/               pytest against in-memory Mongo
│   └── Dockerfile           python:3.12-slim + onnxruntime
├── ml/                      model code, trained on Colab
│   ├── unet.py  dataset.py  train.py  export.py  infer.py  heuristic.py  geo.py
│   ├── weights/unet_sar.onnx metrics.json
│   ├── samples/             4 Sentinel-1 tiles + bboxes.json
│   └── notebooks/train_colab.ipynb
├── docker-compose.yml       local Mongo
└── .github/workflows/ci.yml  ruff, pytest, tsc, vite build
```

Build schedule

Ten weeks from 8 September to 16 November 2026, leaving buffer before the grand finale. Two tracks run in parallel from day one because model training is the longest single dependency.

Week 1
8 to 14 Sep

Foundations and dataset request

Repo, CI, Docker Compose, Atlas cluster, Render and Vercel projects wired to empty apps. Request Krestenitis dataset. Write unet.py and dataset.py. Frontend shell with sidebar, routing and MSW mocks for every endpoint.

Done when: hello-world deploys on both hosts, dataset request sent.

Week 2
15 to 21 Sep

Auth and first training run

Users collection, login, JWT, roles, seed script, Login page, RequireAuth. Colab notebook trains for 40 epochs; record mIoU. Export ONNX.

Done when: officer can log in on the deployed URL; metrics.json exists.

Week 3
22 to 28 Sep

Detection end to end

infer.py with onnxruntime, heuristic fallback, geo.py, POST /api/detect, bundled samples, Detection Console with mask overlay and engine badge.

Done when: a bundled tile returns a polygon and area on the deployed console.

Week 4
29 Sep to 5 Oct

Map and drift

Cesium MapView, slick layer, Open-Meteo drift service, three origin ellipses drawn. Incident creation from a detection.

Done when: Create incident on the console lands you on the globe with slick and ellipses.

Week 5
6 to 12 Oct

AIS live and scenario

AISStream collector running on Render, vessels and ais_positions filling, Vessels page and vessel layer on the map. gen_scenario.py with the guilty tanker and its AIS gap.

Done when: real ships move on the globe; the demo scenario loads from seed.

Week 6
13 to 19 Oct

Ranking and Investigation

Candidate query, categorisation, eight features, tiers, ranking stored on the incident. Investigation page with feature bars, track replay and time slider.

Done when: the tanker ranks first with the AIS gap visible in its bar and on the track.

Week 7
20 to 26 Oct

Incident lifecycle, reports, admin

Events, status, inspection request, hashes. Print route and PDF. Admin users, zones, audit log. Dashboard numbers from real collections.

Done when: an officer closes the loop from detection to PDF without touching the database.

Week 8
27 Oct to 2 Nov

Scene ingest

Copernicus query, windowed download, clip, tile, scheduler. Run against a real recent scene over the Gujarat zone.

Done when: a real scene appears in the scenes collection with its tiles processed automatically.

Week 9
3 to 9 Nov

Assistant, polish, tests

Claude drawer with three tools. Empty states, error messages, loading states. Pytest coverage on ranking and drift. Landing page copy and screenshots.

Done when: CI green, no console errors on any page, assistant answers "why is this ship first".

Week 10
10 to 16 Nov

Demo hardening

Rehearse the three-minute script ten times. Offline fallback: seed data and samples work with no external API. Model card, README architecture diagram, pitch deck.

Done when: the demo runs on a phone hotspot without a hitch.

Three-minute demo

1. **0:00** Logged in as admin. Flash the Admin page: users table with the officer account, system health all green. Log out, sign in as the officer. Dashboard. "A slick was picked up on last night's Sentinel-1 pass over the Gujarat lane." Click the alert.
2. **0:20** Detection Console. Show the raw radar tile. Press Run. Mask appears in under a second, polygon traced, 4.2 km^2, confidence 0.91, engine U-Net, mIoU from our test set in the corner.
3. **0:50** Create incident. Globe flies to the slick. Three origin ellipses fade back in time. "This is where the oil was 6, 12 and 24 hours ago, using last night's wind and current."
4. **1:20** Investigation. Eight ships were in those zones. Ranked list. Top one is a tanker: transponder dark for 40 minutes crossing the 12-hour zone, heading matches the slick's axis, draft dropped. Point at each bar.
5. **2:10** Track replay. Scrub the slider. The tanker's track vanishes exactly where the ellipse is, then reappears.
6. **2:35** Ask the assistant "why is this ship first". It reads the same features back in a sentence.
7. **2:50** Mark for inspection, export PDF. "This is what the officer sends to the boarding team. We give leads in hours instead of weeks; the sample match confirms it."

Risks and honest claims

> **Say these things before the judges ask.**
>
> Sentinel-1 revisit over India is 6 to 12 days, so this is a monitoring system, not a live camera. The demo incident's ship tracks are a generated scenario because free historical AIS does not exist for Indian waters; the live feed is real. The drift model is first-order. The system produces leads, and under MARPOL the culprit is confirmed by an oil sample match on board.

RISK	MITIGATION
Dataset access is delayed	Request in week 1. Meanwhile build against a public SAR oil-spill sample set for pipeline plumbing, and swap in Krestenitis for the real training run.
Model mIoU is disappointing	Heuristic fallback keeps the demo alive. Try the ResNet18 encoder variant. Report the number honestly and show the confusion matrix.
Render memory or cold start	onnxruntime not torch, starter plan, health check pinger. Rehearse with the service already warm.
Venue network fails	Everything needed for the script is seeded or bundled. External APIs are only for live vessels and fresh weather, both of which degrade to cached values.
Cesium bundle is heavy	Lazy-load the map route. Landing and login never load Cesium.
Scope creep	The schedule's "done when" lines are the acceptance tests. Nothing else gets built until the week's line is true.
